

Laboratorios Programación 2

Juego de Cartas: la Veintiuna

1. Descripción general del problema

Se desea desarrollar varias clases para implementar un juego popular de cartas llamado la veintiuna (en inglés, el *Blackjack*) y que consiste en que los jugadores intentan alcanzar una puntuación lo más cercana posible a 21 sin pasarse. El Blackjack se juega con baraja francesa (de 52 cartas), pero nosotros lo vamos a hacer con la baraja española de 40 cartas. Además en el juego pueden participar varios jugadores, pero nosotros lo vamos a restringir a un jugador y la banca (el ordenador).

2. Estructura de carpetas y compilación

Para la realización de esta práctica es necesario organizar el código fuente por paquetes. Además se recomienda crear una carpeta para guardar los archivos fuentes y otra para generar los archivos *.class*. A su vez se supone que la librería *tads* se ha descargado de Moodle y se ha colocado en una subcarpeta *lib*. La estructura de carpeta a partir de la carpeta de trabajo queda así:

```
Practica_Veintiuna
+-- bin
|   +- [Carpetas de paquetes y archivos .class]
+-- lib
|   +- tads-20230510.jar
+-- src
    +-- test
    |   +- [Archivos java con los test]
    +-- veintiuna
        +- [Archivos java con la solucion]
```

Si se ha utilizado la distribución de archivos anterior, el comando para compilar todas las clases desde *Practica_Veintiuna* es:

```
javac -sourcepath src -d bin -cp lib/tads-20230510.jar src/test/*.java src/veintiuna/*.java
```

Para ejecutar los test se hace:

```
java -cp bin test.CartaTest
java -cp bin test.BarajaTest
java -cp bin;lib/tads-20230510.jar test.BancaTest
java -cp bin;lib/tads-20230510.jar test.JugadorTest
```

Y para ejecutar el programa principal:

```
java -cp bin;lib/tads-20230510.jar veintiuna.Veintiuna
```

Atención para ejecutar en **Linux** hay que cambiar ; por : en el comando para ejecutar los tests.

3. Especificación de las clases

Todas las clases relacionadas con el juego se van a colocar en un paquete **veintiuna**. Las clases a programar son las siguientes:

3.1. Clase Carta

Representa una carta de la baraja. Los objetos de esta clase deben ser inmutables. Declara los atributos según consideres adecuado.

```
package veintiuna;

public class Carta {

    // Usa estos atributos de clase para manejar los datos de las cartas.
    private static final String[] PALO = { "Oros", "Espadas", "Bastos", "Copas" };
    private static final String[] NUMERO = { "As", "2", "3", "4", "5", "6", "7", "Sota", "Caballo", "Rey" };
    private static final int[] VALOR = { 11, 2, 3, 4, 5, 6, 7, 10, 10, 10 };

    /**
     * Construye una carta de la baraja.
     *
     * Para generar la carta se supone que las cartas se ordenan del As al Rey y que
     * el orden de los palos es Oros, Espadas, Bastos y Copas. Por ejemplo: la carta
     * 0 es el As de Oros, la 10 es el As de Espadas y la 23 es el 4 de Bastos.
     *
     * @param indice, el indice de la carta entre 0 y 39
     */
    public Carta (int indice)

    /**
     * Devuelve el palo de la carta (uno de los valores en PALO).
     *
     * @return el palo de esta carta
     */
    public String palo ()

    /**
     * Devuelve el numero de la carta (uno de los valores en NUMERO).
     *
     * @return el numero de esta carta
     */
    public String numero ()

    /**
     * Devuelve el numero de la carta (uno de los valores en VALOR).
     *
     * @return el valor de la carta en el juego de la veintiuna
     */
    public int valor ()

    /**
     * Devuelve el nombre de la carta en la forma 'numero' de 'palo'.
     */
    @Override
    public String toString ()
```

3.2. Clase Baraja

Representa una baraja española de 40 cartas. Se puede barajar y se pueden robar cartas de la baraja (se reduce el número de cartas en la baraja). No están declarados todos los atributos necesarios.

```
package veintiuna;

import java.util.Random;

public class Baraja {

    private Carta[] cards;
```

```

private Random rand;

/**
 * Construye una baraja espanhola de 40 cartas.
 *
 * @param rand, el generador de numeros aleatorios
 */
public Baraja (Random rand)

/**
 * Barajar esta baraja.
 *
 * Se emplea el siguiente algoritmo: a) Genera dos enteros aleatorios entre 0 y
 * size()-1 b) Intercambia las cartas en las posiciones dadas por los enteros
 * aleatorios c) Repite a) y b) 'n' veces Es obligatorio usar rand para generar
 * los enteros aleatorios.
 *
 * @param n, las veces que se intercambian las cartas.
 */
public void barajar (int n)

/**
 * Roba una carta de esta baraja.
 *
 * Se debe roba la ultima carta de la baraja.
 *
 * @POST size() se decrementa en 1.
 * @return La carta robada.
 */
public Carta robar ()

/**
 * Numero de cartas en esta baraja.
 *
 * @return el numero de cartas
 */
public int size ()
}

```

3.3. Clase Jugador

El jugador de la Veintiuna. Tiene una mano de cartas y puede robar cartas de la baraja. Puedes declarar más atributos si lo consideras necesario.

```

package veintiuna;

import java.util.Scanner;

import tads.ArrayList;
import tads.IList;

public class Jugador {
    private static final String TEXTO_PREGUNTA = "Robas_o_te_plantas?(carta/me_planto)";
    private static final String TEXTO_ACCION_INVALIDA = "Escribe_'carta'_o_'me_planto'_(sin_comillas)";

    protected IList<Carta> hand;

    /**
     * Construye un jugador. Inicialmente el jugador no tiene ninguna carta.
     */
    public Jugador ()

```

```

/**
 * Este jugador roba una carta de la baraja y la pone en su mano.
 *
 * @param baraja, baraja de donde se roba la carta.
 */
public void robaCarta (Baraja baraja)

/**
 * Devuelve la puntuacion del jugador
 */
public int puntuacion ()

/**
 * Devuelve un texto con las cartas de la mano de este jugador. El texto es: 'Tu
 * mano:', un salto de linea y cada carta en la mano del jugador en una linea de
 * texto. Y finalmente: 'Puntuacion: ', la puntuacion y un salto de linea.
 */
public String muestraMano ()

/**
 * Este jugador hace su jugada.
 *
 * Se pide al usuario si roba o se planta (se escribe en la terminal
 * 'TEXTO_PREGUNTA'). La accion del jugador se lee de scanner (una linea). Si
 * pide "carta" se debe robar una carta y mostrar la mano del jugador. Si el
 * usuario no proporciona una accion valida se debe pedir de nuevo con
 * 'TEXTO_ACCION_INVALIDA'.
 *
 * Se repite hasta que la puntuacion sea mayor igual que 21 o el jugador diga
 * "me planto".
 *
 * @param scanner, las acciones del jugador se leen desde este parametro
 * @param baraja, baraja de donde se roban las cartas.
 */
public void juegaTurnos (Scanner scanner, Baraja baraja)
}

```

3.4. Clase Banca

La banca es un jugador especial que gana en caso de empate y que juega con una estrategia prefijada, roba carta siempre por dejado de 17 y se planta siempre con 17 ó más. Además juega primero, pero sólo muestra la primera carta que recibe. Tienes que declarar los atributos de esta clase.

```
package veintiuna;
```

```
import tads.*;
```

```
public class Banca {
```

```

/**
 * El jugador banca comienza robando una carta.
 *
 * @param baraja, baraja de donde se roba la carta.
 */
public Banca (Baraja baraja)

```

```

/**
 * Este jugador roba una carta de la baraja y la pone en su mano.
 *
 * @param baraja, baraja de donde se roba la carta.
 */
public void robaCarta (Baraja baraja)

```

```
/**
```

```

    * El jugador banca juega primero y muestra solo la primera carta que roba.
    * Debes escribir "Banca:", la primera carta y el numero de cartas que tiene en
    * la mano.
    */
public void muestraPrimeraCarta ()

/**
 * Escribe las cartas de la mano de este jugador 'Banca'. Se debe escribir:
 * 'Banca:' y cada carta en la mano del jugador Banca en una linea de texto. Y
 * finalmente: 'Puntuacion: ' y la puntuacion.
 */
public void muestraMano ()

/*
 * Devuelve la puntuacion este jugador banca
 */
public int puntuacion ()

/**
 * Este jugador banca realiza su jugada.
 *
 * La banca siempre roba carta por debajo de una puntuacion de 17 y se planta
 * con 17 o mas.
 *
 * @param baraja, la baraja de donde roba las cartas
 */
public void juegaTurnos (Baraja baraja)
}

```

4. Pruebas unitarias

Se supone que todas las clases relacionadas con las pruebas se colocan en un paquete **test**. Para comprobar cada una de las clases se pueden utilizar las siguientes pruebas automáticas.

```

package test;

public class Assert {
    private Assert () {
    }

    public static void assertTrue (boolean c) {
        if (!c)
            throw new RuntimeException("\nERROR:_la_condicion_no_es_cierta");
    }

    public static void assertEquals (Object a, Object b) {
        if (a != b)
            throw new RuntimeException(String.format("\nERROR:_Los_objetos_0x%08x_y_0x%08x_NO_son_el_MISMO_objeto",
                System.identityHashCode(a), System.identityHashCode(b)));
    }

    public static void assertNotEq (Object a, Object b) {
        if (a == b)
            throw new RuntimeException(String.format("\nERROR:_Los_objetos_0x%08x_y_0x%08x_SON_el_MISMO_objeto",
                System.identityHashCode(a), System.identityHashCode(b)));
    }

    public static void assertEquals (Object a, Object b) {
        if (!a.equals(b))
            throw new RuntimeException(String.format("\nERROR:_Los_objetos_\n%s\_(0x%08x)_y_\n%s\_(0x%08x)_NO_son_IGUALES",
                a, System.identityHashCode(a), b, System.identityHashCode(b)));
    }
}

```

```

public static void assertEquals (Object a, Object b) {
    if (a.equals(b))
        throw new RuntimeException(String.format("\nERROR: Los objetos \" %s \"_(0x%08x)_y_\" %s \"_(0x%08x)_SON_IGUALES", a,
            System.identityHashCode(a), b, System.identityHashCode(b)));
}

}

package test;

import veintiuna.Carta;

public class CartaTest {

    // Usa estos atributos de clase para manejar los datos de las cartas.
    private static final String[] PALO = { "Oros", "Espadas", "Bastos", "Copas" };
    private static final String[] NUMERO = { "As", "2", "3", "4", "5", "6", "7", "Sota", "Caballo", "Rey" };
    private static final int[] VALOR = { 11, 2, 3, 4, 5, 6, 7, 10, 10, 10 };

    static void test1 () {
        Carta c = new Carta(0); // As de Oros
        Assert.assertEquals(c.numero(), NUMERO[0]);
        Assert.assertEquals(c.palo(), PALO[0]);
        Assert.assertEquals(c.valor(), VALOR[0]);
        Assert.assertEquals(c.toString(), "As_de_Oros");
    }

    static void test2 () {
        Carta c = new Carta(11); // Dos de Espadas
        Assert.assertEquals(c.numero(), NUMERO[1]);
        Assert.assertEquals(c.palo(), PALO[1]);
        Assert.assertEquals(c.valor(), VALOR[1]);
        Assert.assertEquals(c.toString(), "2_de_Espadas");
    }

    static void test3 () {
        Carta c = new Carta(27); // Sota de Bastos
        Assert.assertEquals(c.numero(), NUMERO[7]);
        Assert.assertEquals(c.palo(), PALO[2]);
        Assert.assertEquals(c.valor(), VALOR[7]);
        Assert.assertEquals(c.toString(), "Sota_de_Bastos");
    }

    static void test4 () {
        Carta c = new Carta(38); // Caballo de Copas
        Assert.assertEquals(c.numero(), NUMERO[8]);
        Assert.assertEquals(c.palo(), PALO[3]);
        Assert.assertEquals(c.valor(), VALOR[8]);
        Assert.assertEquals(c.toString(), "Caballo_de_Copas");
    }

    static void test5 () {
        Carta c = new Carta(9); // Rey de Oros
        Assert.assertEquals(c.numero(), NUMERO[9]);
        Assert.assertEquals(c.palo(), PALO[0]);
        Assert.assertEquals(c.valor(), VALOR[9]);
        Assert.assertEquals(c.toString(), "Rey_de_Oros");
    }

    public static void main (String[] args) {
        test1();
        test2();
        test3();
        test4();
        test5();
    }
}

```

```

        System.out.println("Han_pasado_todas_las_pruebas_de_Carta");
    }
}

package test;

import java.util.Random;

import veintiuna.*;

public class BarajaTest {

    static void test1 () {
        Random r = new Random(1122); // Se generan los mismos numeros
        Baraja b = new Baraja(r);
        Assert.assertEquals(b.size(), 40);
        for (int i = 39; i >= 0; --i) {
            Carta u = b.robar();
            Carta esperada = new Carta(i);
            Assert.assertEquals(u.toString(), esperada.toString());
            Assert.assertEquals(b.size(), i);
        }
    }

    static void test2 () {
        Random r = new Random(1122);
        // Se muestran los primeros 20 enteros aleatorios generados
        // (Para mostrar lo que se va esperar)
        System.out.println("Mostrando_numeros_aleatorios");
        for (int i = 0; i < 10; ++i) {
            System.out.println(r.nextInt(40) + "_" + r.nextInt(40));
        }
    }

    static void test3 () {
        // Se han hecho los intercambios mostrados antes
        int[] indice = { 0, 15, 11, 4, 21, 5, 6, 22, 8, 3, // Hasta 10
            10, 2, 12, 13, 14, 1, 23, 17, 18, 36, // De 11 a 20
            20, 9, 28, 16, 24, 25, 26, 19, 7, 29, // De 21 a 30
            30, 31, 32, 33, 34, 35, 27, 37, 38, 39 // De 31 a 40
        };
        Random r = new Random(1122); // Se generan los mismos numeros
        Baraja b = new Baraja(r);
        b.barajar(10);
        for (int i = 39; i >= 0; --i) {
            Carta u = b.robar();
            Carta esperada = new Carta(indice[i]);
            Assert.assertEquals(u.toString(), esperada.toString());
            Assert.assertEquals(b.size(), i);
        }
    }

    public static void main (String[] args) {
        test1();
        test2();
        test3();
        System.out.println("Han_pasado_todas_las_pruebas_de_Baraja");
    }
}

package test;

import java.util.Random;
import java.util.Scanner;

```

```

import veintiuna.*;

public class JugadorTest {

    static void test1 () {
        Jugador j = new Jugador();
        Random r = new Random(1122); // Se generan los mismos numeros
        Baraja b = new Baraja(r);
        Assert.assertEquals(j.muestraMano(), "Tu_mano:\nPuntuacion:_0\n");
        j.ropaCarta(b);
        Assert.assertEquals(j.muestraMano(), "Tu_mano:\nRey_de_Copas\nPuntuacion:_10\n");
        j.ropaCarta(b);
        Assert.assertEquals(j.muestraMano(), "Tu_mano:\nRey_de_Copas\nCaballo_de_Copas\nPuntuacion:_20\n");
        j.ropaCarta(b);
        Assert.assertEquals(j.muestraMano(), "Tu_mano:\nRey_de_Copas\nCaballo_de_Copas\nSota_de_Copas\nPuntuacion:_30\n");
    }

    static void test2 () {
        // Comprobar puntuacion con As (con un solo As).
        Jugador j = new Jugador();
        Random r = new Random(1122); // Se generan los mismos numeros
        Baraja b = new Baraja(r);
        // Quito 9 cartas.
        for (int i = 0; i < 9; ++i) {
            b.robear();
        }
        j.ropaCarta(b); // Robo As de Copas
        Assert.assertEquals(j.muestraMano(), "Tu_mano:\nAs_de_Copas\nPuntuacion:_11\n");
        j.ropaCarta(b); // Robo Rey de Espadas
        Assert.assertEquals(j.puntuacion(), 21);
        j.ropaCarta(b); // Robo Caballo de Espadas
        Assert.assertEquals(j.puntuacion(), 21);
    }

    static void test3 () {
        // Comprobar puntuacion con As (con dos ases).
        Jugador j = new Jugador();
        Random r = new Random(1122); // Se generan los mismos numeros
        Baraja b = new Baraja(r);
        // Quito 9 cartas.
        for (int i = 0; i < 9; ++i) {
            b.robear();
        }
        j.ropaCarta(b); // Robo As de Copas
        Assert.assertEquals(j.puntuacion(), 11);
        // Quito 9 cartas.
        for (int i = 0; i < 9; ++i) {
            b.robear();
        }
        j.ropaCarta(b); // Robo As de Espadas
        Assert.assertEquals(j.puntuacion(), 2);
        // Quito 3 cartas (figuras).
        for (int i = 0; i < 3; ++i) {
            b.robear();
        }
        j.ropaCarta(b); // Robo el 7
        Assert.assertEquals(j.puntuacion(), 9);
        j.ropaCarta(b); // Robo el 6
        Assert.assertEquals(j.puntuacion(), 15);
        j.ropaCarta(b); // Robo el 5
        Assert.assertEquals(j.puntuacion(), 20);
        // Quito 4, 3, 2.
        for (int i = 0; i < 3; ++i) {
            b.robear();
        }
    }
}

```



```

    }
    j.robaCarta(b); // Robo el as
    Assert.assertEquals(j.puntuacion(), 21);
    j.robaCarta(b); // Robo el Rey
    Assert.assertEquals(j.puntuacion(), 31);
}

static void test4 () {
    System.out.println("---_test_4_---");
    System.out.println("->_Se_van_a_mostrar_los_textos_que_se_muestran_al_usuario,_pero_no_se_comprueban");
    String input = "carta\ncarta\ncarta\ncarta\ncarta\ncarta\n";
    Scanner s = new Scanner(input);
    Random r = new Random(1122); // Se generan los mismos numeros
    Baraja b = new Baraja(r);
    b.barajar(10);
    // Quito 3 cartas.
    for (int i = 0; i < 3; ++i) {
        b.robar();
    }
    Jugador j = new Jugador();
    j.juegaTurnos(s, b);
    s.close();
    Assert.assertEquals(j.puntuacion(), 21);
}

static void test5 () {
    System.out.println("---_test_5_---");
    System.out.println("->_Se_van_a_mostrar_los_textos_que_se_muestran_al_usuario,_pero_no_se_comprueban");
    String input = "carta\ncarta\ncarta\ncarta\ncarta\ncarta\n";
    Scanner s = new Scanner(input);
    Random r = new Random(1122); // Se generan los mismos numeros
    Baraja b = new Baraja(r);
    b.barajar(10);
    Jugador j = new Jugador();
    j.juegaTurnos(s, b);
    Assert.assertEquals(j.puntuacion(), 30);
}

// Se planta en 20
static void test6 () {
    System.out.println("---_test_6_---");
    System.out.println("->_Se_van_a_mostrar_los_textos_que_se_muestran_al_usuario,_pero_no_se_comprueban");
    String input = "carta\ncarta\nusuario_se_equivoca\nme_planto\n"; // Una de las veces el usuario se equivoca
    Scanner s = new Scanner(input);
    Random r = new Random(1122); // Se generan los mismos numeros
    Baraja b = new Baraja(r);
    b.barajar(10);
    Jugador j = new Jugador();
    j.juegaTurnos(s, b);
    Assert.assertEquals(j.puntuacion(), 20);
}

public static void main (String[] args) {
    test1();
    test2();
    test3();
    test4();
    test5();
    test6();
    System.out.println("---_---_");
    System.out.println("Han_pasado_todas_las_pruebas_de_Jugador");
}
}

package test;

```

```

import java.util.Random;

import veintiuna.*;

public class BancaTest {

    static void test1 () {
        Random r = new Random(1122); // Se generan los mismos numeros
        Baraja baraja = new Baraja(r);
        Banca banca = new Banca(baraja);
        Assert.assertEquals(banca.puntuacion(), 10);
        banca.juegaTurnos(baraja);
        Assert.assertEquals(banca.puntuacion(), 20);
    }

    static void test2 () {
        Random r = new Random(1122); // Se generan los mismos numeros
        Baraja baraja = new Baraja(r);
        for (int i = 0; i < 9; ++i) {
            baraja.robar();
        }
        Banca banca = new Banca(baraja);
        Assert.assertEquals(banca.puntuacion(), 11);
        banca.juegaTurnos(baraja);
        Assert.assertEquals(banca.puntuacion(), 21);
    }

    static void test3 () {
        Random r = new Random(1122); // Se generan los mismos numeros
        Baraja baraja = new Baraja(r);
        baraja.barajar(10);
        for (int i = 0; i < 9; ++i) {
            baraja.robar();
        }
        Banca banca = new Banca(baraja);
        Assert.assertEquals(banca.puntuacion(), 11);
        banca.juegaTurnos(baraja);
        Assert.assertEquals(banca.puntuacion(), 21);
    }

    static void test4 () {
        // 20 Test aleatorios "fijos"
        Random r = new Random(2313);
        for ( int i = 0; i < 20; ++i ) {
            Baraja baraja = new Baraja(r);
            baraja.barajar(100);
            Banca banca = new Banca(baraja);
            banca.juegaTurnos(baraja);
            Assert.assertTrue(banca.puntuacion() >= 17);
        }
    }

    public static void main (String[] args) {
        test1();
        test2();
        test3();
        test4();
        System.out.println("Han pasado todas las pruebas de Banca");
    }
}

```

5. Programa final

Una vez desarrolladas todas las partes del juego, se puede montar una clase que integre todas las clases anteriores para generar el juego. Puedes utilizar esta que sirve para dos jugadores: una persona y la banca o puedes hacer la tuya propia (por ejemplo, si quieres que sirva para más jugadores).

```
package veintiuna;

import java.util.Random;
import java.util.Scanner;

public class Veintiuna {
    private Random r = new Random();
    private Jugador jugador = new Jugador();
    private Banca banca;
    private Baraja baraja;

    Veintiuna () {
        baraja = new Baraja(r);
        banca = new Banca(baraja);
    }

    void jugar (Scanner scanner) {
        baraja.barajar(100);
        banca.juegaTurnos(baraja);
        System.out.println("La_banca_ha_jugado_y_tiene:");
        banca.muestraPrimeraCarta();
        System.out.println("Ahora_es_el_turno_del_jugador:");
        jugador.juegaTurnos(scanner, baraja);
        muestraGanador();
    }

    void muestraGanador () {
        System.out.println("\nLa_mano_final_del_Jugador:");
        System.out.println(jugador.muestraMano());
        System.out.println("\nLa_mano_final_de_la_Banca:");
        banca.muestraMano();
        int puntuacion = jugador.puntuacion();
        int puntuacionBanca = banca.puntuacion();
        if (puntuacion > 21) {
            System.out.println("El_jugador_se_ha_pasado,_la_banca_gana");
        } else if (puntuacionBanca > 21) {
            System.out.println("La_banca_se_ha_pasado,_el_jugador_gana");
        } else if (puntuacion > puntuacionBanca) {
            System.out.println("Jugador_gana");
        } else {
            System.out.println("La_banca_gana");
        }
    }

    public static void main (String[] args) {
        Veintiuna juego = new Veintiuna();
        Scanner scanner = null;
        try {
            scanner = new Scanner(System.in);
            juego.jugar(scanner);
        } catch (Exception e) {
            e.printStackTrace();
            System.out.println(e);
        }
        scanner.close();
    }
}
```